

NLP Project

The Local RAG & Deployment Challenge

(CV Matching System)

Dr. Sally Saad

Team ID: 32

Level	Department	ID	Name	
4	Scientific Computing(SC)	2022170006	ابراهيم محمد ابراهيم احمد	1
		2022170445	منه حسن محمد حسن	2
		2022170159	روان محمد حمدي سعد	3
		2022170604	روى حسين محمد عبدالرحيم	4
		2022170268	عمر احمد مدحت الحكيم	5
		2022170269	عمر احمد وطني ثابت	6

2025 – 2026

Phase 1: Data Processing & Vectorization

1. Project Overview

The **CV Matcher** is an intelligent recruitment solution designed to streamline the hiring process using Natural Language Processing (NLP) and Retrieval-Augmented Generation (RAG). The system enables recruiters to input a **Job Description (JD)** and automatically retrieve the most semantically relevant resumes from a massive database. Additionally, it supports a "**Cold Start**" feature where users can upload their own resumes in various formats (PDF, Word, Images) to check their compatibility against specific job roles.

2. Phase 1.1: Data Acquisition & Dataset Scaling

In this phase, we constructed a robust "**Gold Standard**" dataset to ensure high-quality model training and retrieval.

- **Research Question:** We formulated a querying problem where an unstructured JD is used to find the most semantically similar resumes, moving beyond simple keyword matching to deep semantic search.
- **Dataset Volume:** Successfully processed and consolidated **24,257** resumes.
- **Diverse Sources:** Integrated data from six different Kaggle sources to ensure a wide representation of industries such as Accounting, Engineering, IT, and Healthcare.
- **Technical Challenge:** We managed a heterogeneous environment containing plain text, digital PDFs, and scanned documents requiring optical character recognition (OCR).

3. Phase 1.2: Universal CV Reader Development (Extraction Strategy)

A core component of Phase 1 was the development of the `universal_cv_reader` module, which implements an **Extraction Strategy** to handle unstructured file ingestion:

- **PDF Processing:** Utilized the `PyMuPDF` (`fitz`) library for direct text extraction. We implemented a fallback mechanism: if the extracted text is less than 50 characters, the system treats the PDF as a "scanned document" and triggers the OCR engine.
- **OCR Integration:** Integrated the `Tesseract` OCR engine to process image-based formats (JPG, PNG, WebP), converting visual data into machine-readable text.
- **Word Documents:** Used the `python-docx` library to parse structured data from `.docx` files.

4. Phase 1.3: Advanced NLP Pre-processing Pipeline

To transform raw text into high-quality data for the Embedding Model, we designed a multi-stage cleaning pipeline:

1. **Noise Removal:** Stripped URLs, email addresses, and non-informative special characters using Regular Expressions.
2. **PII Masking & Anonymization:** Developed a custom RegEx script to identify and remove phone numbers (sequences of 7 to 15 digits). This enhances privacy and reduces numerical noise during vectorization.
3. **Category Standardization:** Addressed the issue of redundant categories by mapping 85 raw labels into **47 distinct, high-level job categories** (e.g., merging "Java Developer" and "Javadeveloper").
4. **Normalization:** Applied case folding (lowercasing) and whitespace normalization to ensure uniform tokenization.

5. Phase 1.4: Data Quality Assurance (QA) & Cleaning

Before finalizing the dataset, we applied rigorous filtering to optimize the system's performance:

- **Deduplication:** Identified and removed **3,349 duplicate records** based on text similarity. This resulted in a unique pool of **22,485** (later refined to **22,160**) resumes.
- **Length Filtering:** Excluded resumes with less than 100 characters or fewer than 15 words. This ensures that the vector database only contains documents with sufficient semantic context for accurate matching.

6. Phase 1.5: Chunking Strategy & Semantic Search Pipeline

Once the text is parsed and cleaned, the system applies a professional RAG pipeline to prepare the data for semantic search. This stage is governed by the following technical configurations:

- **Pipeline Configuration:**
 - **Text Splitter:** RecursiveCharacterTextSplitter (LangChain).
 - **Embedding Model:** all-MiniLM-L6-v2 (SentenceTransformers).
 - **Vector Store:** ChromaDB — Persistent Local Client.
 - **Chunk Settings:** Size = 500 characters, Overlap = 125 characters (25%).
 - **Separators:** Space (" ") and empty string ("") — non-regex.
 - **ID Format:** cv_{row_index}_chunk_{chunk_index} for precise traceback.
 - **Metadata Fields:** candidate_id, category.
- **Chunking Strategy — Comparative Analysis:** Four chunking configurations were evaluated to ensure optimal retrieval performance. The following table summarizes the findings:

Chunk Size	Overlap	Status	Distance Range	Observed Issue	Verdict
Small (<200)	~50	Failed	0.75 - 0.90	Chunks too small, context lost across boundaries; embeddings lacked enough semantic signal.	Rejected
500	125	Tested	0.40 – 0.55	None observed; stable and consistent distances across top-3 results.	Selected
800	200	Tested	0.49 – 0.56	Similar distance range to 500; no meaningful improvement.	Acceptable
1500	375	Failed	Inconsistent	Chunks exceeded embedding model context window; semantic drift, noisy distances.	Rejected

- **Detailed Justification:**

- **500 chars (Selected):** Produced semantically rich vectors. Top-3 results showed narrow distance ranges (0.5457 to 0.5534), confirming reliable relevance.
 - **1500 chars (Rejected):** Exceeded the effective context window of `all-MiniLM-L6-v2` (optimized for ~256 tokens), leading to inconsistent similarity scores.
- **3.1 Small Chunks (< 200 chars):** Chunks were too small to carry meaningful semantic context. Each chunk contained only a few keywords, causing the embedding model to produce weak, noisy vectors that failed to represent the candidate's actual skill set. This led to unreliable similarity scores and poor ranking.
- **3.2 Chunk Size 500 — Selected:** A chunk size of 500 with an overlap of 125 (25%) produced stable and semantically rich vectors. The top-3 results returned consistent distances in the range 0.5457 to 0.5534, confirming strong and reliable relevance. This configuration was adopted as the final setting.
- **3.3 Chunk Size 800:** Increasing to 800 characters with a 200-character overlap produced results in a similar distance range (approx. 0.49 to 0.56). While acceptable, there was no measurable improvement over 500, making it unnecessary for this use case.
- **3.4 Chunk Size 1500 — Rejected:** At 1500 characters, chunks frequently exceeded the effective context window of the `all-MiniLM-L6-v2` model (optimized for inputs up to ~256 tokens). This caused semantic drift: the model averaged across too much unrelated content, degrading embedding quality.

7. Phase 1.6: Sample Query & Search Results

To validate the architecture, a sample query was executed against the "Data Science" category.

- **Job Description:** "Data scientist proficient in Python, SQL, and data visualization using Tableau."

Rank	Candidate ID	Distance Score	Snippet Preview
#1	CV_2282	0.5457	<i>coding skills in python numpy pandas and machine learning related libraries... effective data visualization with matplotlib, bokeh, seaborn, qliksense and tableau...</i>
#2	CV_15564	0.5499	<i>skills in python numpy pandas and machine learning related libraries... effective data visualization with matplotlib, bokeh, seaborn, qliksense and tableau...</i>
#3	CV_15464	0.5534	<i>coding skills in python numpy pandas and machine learning related libraries... effective data visualization with matplotlib, bokeh, seaborn, qliksense and tableau...</i>

- **Results Discussion:** All three returned candidates show strong alignment with the query. The narrow distance range (0.5457 – 0.5534) indicates consistent retrieval quality, with no significant drop-off between ranks.
- **Key Observations:**
 - **Category Filter:** The filter `where={"category": target_category}` effectively scoped the search, reducing noise from unrelated fields.
 - **Overlap Continuity:** The 25% overlap ensures continuity across chunk boundaries, preserving multi-token skill phrases that might otherwise be split.
 - **ID-based Traceback:** The system (`cv_{index}_chunk_{i}`) enables precise traceback from any retrieved chunk to its source resume.
 - **Idempotent Logic:** The `collection.count() == 0` check prevents duplicate embeddings on re-runs, making the pipeline production-safe.
- **Conclusion:** The configuration of **chunk_size=500** with **chunk_overlap=125**, combined with the `all-MiniLM-L6-v2` model and **ChromaDB**, delivers a stable and accurate semantic CV retrieval system. Smaller chunks lacked context; larger chunks degraded quality. This mid-range setting is recommended for production deployment.

8. Phase 1.7: Final Output & Tech Stack

- **Final Output:** A high-fidelity Master CSV and a populated ChromaDB vector store containing cleaned, chunked, and vectorized resumes.
- **Tech Stack:** Python, Pandas, PyMuPDF, Tesseract OCR, python-docx, RegEx, LangChain, and ChromaDB.

- **Next Milestone:** The system is now fully prepared for **Phase 2**, which involves building the FastAPI backend and orchestrating the RAG retrieval logic.

Phase 2: System Architecture (FastAPI & Docker)

1. Robust API Interface Development

In this phase, we developed a high-performance backend using the **FastAPI** framework to serve as the interface for our RAG logic. FastAPI was selected for its native support for asynchronous programming, which is critical for handling long-running AI inference tasks and large-scale vector database queries efficiently.

2. Clean Software Engineering: The MVC Pattern

To ensure maintainability and scalability, we strictly applied the **Model-View-Controller (MVC)** design pattern to decouple system concerns:

- **Models (core/ & db/):** Defines the data structures and the lifecycle of the Vector Database (ChromaDB) and LLM configurations.
- **Controllers/Services (services/):** Contains the core **RAG Logic**. This layer orchestrates the communication between the embedding service, the retrieval from ChromaDB, and the context injection into the LLM.
- **Views/Routes (api/):** Manages the API endpoints (Routing). It handles incoming requests, such as CV uploads and JD queries, and returns structured JSON responses to the client.

3. System Components & Modular Pipeline

The system is built on a modular "Retrieval-Generation" pipeline consisting of the following core layers:

3.1. Semantic Retrieval Layer (The Context)

- **Vector Database: ChromaDB** is used to store and manage over 247,000 CV embeddings.
- **Embedding Model:** We utilized **all-MiniLM-L6-v2** to transform unstructured job descriptions and CVs into dense vectors.
- **Retrieval Logic:** Upon receiving a user query, the system performs a similarity search to retrieve the top N candidates whose skill sets align semantically with the Job Description (JD).

3.2. Generation Layer (LLM Logic)

- **Model Selection:** **Gemini 3.1 Flash-Lite** was chosen for its exceptional balance between low latency and high reasoning capabilities. Its "Lite" architecture allows for rapid inference, making it ideal for processing multiple candidate evaluations simultaneously.
- **LLM Integration:** The system connects via the **google-generative SDK**. To ensure stability, the API version v1 is explicitly targeted to avoid deprecation issues.
- **LLM Factory:** We implemented a factory pattern within the services layer to switch seamlessly between different LLM providers (e.g., Gemini API or local Ollama) based on environment variables.

4. Implementation Details: Dynamic Prompt Engineering

A sophisticated "**System Instruction**" framework is used to guide the LLM. The prompt is structured into three sections:

1. **Persona:** Acting as a "**Technical HR Specialist**".
2. **Context Injection:** Inserting the retrieved CVs directly into the prompt.
3. **Output Constraints:** Requiring a structured report that includes "**Key Strengths**" and "**Missing Skill Gaps**".

5. Containerization & Deployment

To guarantee that the **CV Matcher** runs consistently across different development and production environments, the entire application was packaged using **Docker**:

- **Dockerization:** The backend (FastAPI), RAG logic, and dependencies (including Tesseract OCR and Poppler) are encapsulated in a Docker container.
- **Docker-Compose:** We utilized a `docker-compose.yml` file to orchestrate the multi-container ecosystem, including the FastAPI Backend service, the ChromaDB vector store for persistent data, and an optional Ollama service for running local LLMs.
- **Environment Management:** Sensitive data, including the Gemini API Key, is managed through environment variables (`.env`), adhering to security best practices for cloud-based AI applications.

6. API Documentation & Diagnostics

FastAPI's built-in support for **Swagger UI** was utilized to provide interactive API documentation. This allows for real-time testing of the RAG pipeline, monitoring of the VectorDB's health, and debugging of the semantic search results through diagnostic endpoints.

Phase 3: The RAG Logic & Retrieval

1. Implementation of the Querying Pipeline

The querying pipeline is the heart of the RAG system, where the user's Job Description (JD) is matched against the stored CVs. The implementation follows a structured sequence to ensure accurate retrieval and generation:

- **API Request Ingestion:** The FastAPI backend provides a dedicated endpoint (e.g., `/api/v1/cv_matcher/search`) that accepts a user query or a full Job Description in JSON format.
- **Query Embedding:** Upon receiving the text, the system utilizes the `EmbeddingService` to transform the unstructured query into a dense 384-dimensional vector using the `all-MiniLM-L6-v2` model.
- **Vector Similarity Search:** The generated query vector is sent to **ChromaDB**, where a semantic similarity search (using Cosine or L2 distance) is performed against the collection of CV embeddings.
- **Top-k Retrieval:** The system identifies and retrieves the **top-k** (the most relevant) chunks of text from various CVs that best align with the requirements of the Job Description.

2. Context Injection and LLM Generation

Once the relevant documents are retrieved, the "Augmentation" and "Generation" phases take place:

- **Context Window Injection:** The retrieved CV chunks, along with their associated metadata (like filenames), are compiled into a single string known as `context_docs`. This context is then injected directly into a highly engineered "System Instruction" prompt.
- **Dynamic Prompting:** The system uses a **Technical HR Specialist** persona to guide the LLM. The prompt forces the model to rely strictly on the provided context, which significantly reduces the risk of AI hallucinations.
- **LLM Processing:** The combined prompt (Context + Job Description) is sent to the LLM (Gemini 3.1 Flash-Lite or a local Ollama model) via the **LLM Factory Pattern**.
- **Structured API Response:** The LLM generates a structured HR evaluation report, highlighting "Key Strengths" and "Missing Skill Gaps". This generated answer is then returned to the client as a clean JSON response via the API endpoint.

3. Engineering Justification

- **Efficiency:** By retrieving only the most relevant segments (chunks) rather than entire documents, the system stays within the LLM's token window limits and reduces inference latency.

- **Accuracy:** The RAG architecture ensures that the LLM's reasoning is grounded in the actual data stored in the vector database, providing a factual basis for candidate assessments.

Phase 4: Evaluation & Error Analysis

1. Retrieval Accuracy Assessment

Evaluating the system's performance is critical, as RAG architectures are inherently susceptible to retrieval errors and hallucinations. Our evaluation reveals a system that shows resilience in handling complex or "messy" inputs but currently struggles with core retrieval precision and ranking accuracy. While the system can identify relevant candidates, it often fails to rank them correctly or misses the most qualified individuals due to architectural limitations.

2. Edge-Case Analysis and System Failures

The following three edge cases highlight specific scenarios where the system's architecture missed the mark:

- **Case 1: Precision and Ranking Failure (The "Top Candidate" Miss)**
 - **Failure:** In queries for highly specific roles, such as a "Senior Java Developer," the system failed to rank the most qualified candidate at the top, often placing less relevant profiles in the primary spot.
 - **Architectural Reason:** This is primarily due to the **Vector Similarity Gap**. Relying solely on a single-stage dense retrieval (Cosine Similarity) is often insufficient for distinguishing between high-level seniority and general skill matches. The architecture lacks a **Re-ranking Stage** (such as a Cross-Encoder) to refine the initial retrieval results.
- **Case 2: Semantic Noise and "Messy" Input Handling**
 - **Failure:** The system successfully retrieved chunks from documents with non-standard formatting or noisy text, but the LLM struggled to extract a coherent evaluation from these "messy" retrieved segments.
 - **Architectural Reason:** The **Chunking Strategy** did not adequately account for structural noise in unstructured data. When the `universal_cv_reader` passes noisy text to the embedding model, the resulting vectors can be diluted, leading to the retrieval of chunks that contain the right keywords but lack the necessary semantic clarity for the LLM to process effectively.
- **Case 3: Hallucination despite Contextual Injection**
 - **Failure:** In cases where a candidate had a "gap" in a specific required skill, the LLM occasionally inferred or "hallucinated" that the candidate possessed the skill based on related but distinct experience in the retrieved context.
 - **Architectural Reason:** This stems from **Contextual Over-Reliance** combined with a lack of **Strict Negative Constraints** in the system prompt. If the retrieved chunks are semantically close to the query but do

not explicitly confirm a skill, the LLM's generative nature may prioritize "completing the logic" over factual absence, especially without a robust verification layer.

Bonus: Advanced Feature Implementation

Bonus 1: The LLM Factory Pattern Implementation (+5%)

1. Architectural Overview

As part of the advanced engineering requirements, we moved away from hardcoding a single AI model dependency. Instead, we implemented the **Factory Design Pattern** to create a highly decoupled and flexible system. This pattern provides a **Unified Interface** that enables the application to switch between different Large Language Models (LLMs) seamlessly, without modifying the core RAG logic or routing code.

2. Technical Execution

The implementation is structured within the `services/llm/` directory to ensure clean separation of concerns:

- **The Provider Interface:** We developed abstract providers that share a common execution method (e.g., `generate_response`). Currently, the system supports:
 - **GeminiProvider:** Interfaces with remote Google Gemini APIs for high-performance cloud inference.
 - **OllamaProvider:** Interfaces with local models (e.g., Mistral, Qwen) running via Ollama for offline processing.
- **The Factory Logic (LLMService):** A central factory function manages the instantiation of providers based on environment configurations. It dynamically reads the `LLM_PROVIDER` variable from the `.env` file to determine which provider to inject into the RAG pipeline.

3. Engineering & Business Value

Implementing the Factory Pattern elevates the project to **Production-Standard** software through the following benefits:

- **Single Configuration Change:** Switching between a cloud-based API (Gemini) and a local, privacy-focused model (Ollama/Mistral) requires only a one-line change in the environment settings, with zero downtime or code refactoring.
- **System Resilience & Cost Optimization:** In the event of an external API outage or to reduce operational costs, the system can immediately failover to a local model.

- **Future-Proof Scalability:** The architecture is designed for easy expansion. Adding support for other providers (e.g., OpenAI or Anthropic) only requires creating a new provider class and adding it to the factory, ensuring the core "CV Matcher" logic remains untouched.

Bonus 2: Arabic Language Support Implementation **(+10%)**

1. Functional Context Understanding

Instead of relying on a pre-defined English template, the system was tested with a purely colloquial Arabic prompt to evaluate its ability to infer intent and extract parameters dynamically.

2. Practical Test Case & Discussion

We conducted a test case using a casual, real-world Arabic query to see how the RAG pipeline and the LLM would interact with Arabic script.

Test Case Details:

- **User Query (Arabic):** "محتاج ٥ مبرمجين ويب بسرعة يا حبيب اخوك" (I need 5 web developers quickly, my friend).
- **Observation & Results :**
 - **Intent Extraction:** The system successfully understood that the required "**job_description**" was "Web Developer" based on the context of the sentence.
 - **Parameter Identification:** It accurately identified the required number of candidates (**Top 5**) as requested in the query.
 - **Linguistic Matching:** The system retrieved 5 candidates who speak Arabic or have Arabic-based profiles (e.g., "Ahmed Mohamed", "Mohamed Ahmed"), aligning with the language used in the input query.
 - **System Resilience:** Even though the query was written in a non-formal, colloquial way, the system showed high resilience in processing the "messy" input and providing a coherent response.

3. Conclusion

While the Arabic language support was not implemented as a standalone comprehensive NLP module, this test proves that the current architecture—leveraging the **Gemini 3.1 Flash-Lite** reasoning—is highly capable of understanding Arabic intent, extracting job requirements from casual speech, and retrieving linguistically relevant candidates from the database.